

1회차 (2026/8/25)

백엔드 과외 1회차

> 2026-08-25 / 강사: 김승진

> 목표: 프론트엔드 5~6년 → 백엔드(서버) 전향, 12월 말까지 (육아휴직 기간 활용)

0. 커리큘럼 개요

| 구분 | 내용 |

| --- | --- |

| CS 기초 | 운영체제, 자료구조, 알고리즘, 네트워크 |

| 실무 기술 | Java / Spring, 데이터베이스, 메시지큐, 인프라, 클라우드 |

| 심화 | 대용량 트래픽, 스케일아웃, 모니터링 / 로깅 |

| 진행 방식 | 주 1회 · 이론 + 실습 · 과제(프로젝트) |

| 아웃풋 | 포트폴리오 → 면접 준비(코테 포함) → 연봉 협상 → 수업 종료 |

=====

승진님의 개인공부

자바기본/스프링기본

편하게 질문하기!

2주차 예습 키워드

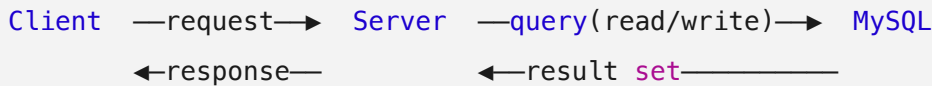
- 스프링 Bean
- 스프링 Bean 생성방법들
- 스프링 Dependency, DI
- Bean 라이프사이클

=====

배운 내용 정리

1. 클라이언트와 서버는 어떻게 주고받는가

가장 단순한 형태. 모든 구조의 출발점.



흐름 정리

1. 클라이언트(브라우저 / 앱)가 HTTP 요청을 보낸다
2. 서버(Spring 애플리케이션)가 요청을 받아 비즈니스 로직 처리
3. 필요한 데이터를 DB에 쿼리 (**read** = SELECT, **write** = INSERT/UPDATE/DELETE)
4. DB가 결과를 서버로 반환
5. 서버가 응답 형태(JSON 등)로 가공해서 클라이언트에 반환

프론트엔드 관점에서의 전환 포인트

- 프론트에서 **fetch** 로 던지던 그 요청의 **오른쪽 끝**이 앞으로 내 담당 영역
- 프론트는 "응답이 왔는지"가 중요하지만, 백엔드는 "**이 요청이 초당 5,000개 오면?**" 이 중요
- 이 그림에서 서버 1대 / DB 1대는 트래픽이 늘어나면 반드시 병목이 됨 → 2번부터가 그 해결 과정

상태(state)의 위치

- 서버는 기본적으로 **stateless** 하게 설계 (요청 간에 서버 메모리에 상태를 남기지 않음)
- 상태는 DB / Redis 같은 외부 저장소로 뺀다
- 이게 지켜져야 4번의 스케일아웃이 가능해짐 → **1번과 4번은 이어지는 얘기**

2. Scale up (수직 확장)

보통은 서버 한대로 시작한다.

하지만 사용자가 늘어나면서 서버가 처리해야되는 요청이 증가하면 어느 순간부터 서버의 CPU나 Memory같은 자원이 부족해질 수 있다.

-
- ```
graph LR; Requests[요청 증가] --> CPU[CPU]; Requests --> Memory[Memory]; Requests --> ThreadPool[Thread Pool]; Requests --> DBConnection[DB Connection]; CPU --> ResourceUsage[등의 자원 사용 증가]; Memory --> ResourceUsage; ThreadPool --> ResourceUsage; DBConnection --> ResourceUsage; ResourceUsage --> Limit[특정 자원이 한계에 가까워짐];
```
- 요청 증가
  - CPU / Memory / Thread Pool / DB Connection 등의 자원 사용 증가
  - 특정 자원이 한계에 가까워짐

- 요청 대기 증가
- 응답 시간 증가 또는 에러 발생

이때 가장 먼저 생각할 수 있는 방법이 **Scale Up**

"서버 대 수는 그대로 두고, 한 서버의 성능을 높이는 것"

AWS라면 인스턴스 타입만 바꾸면 끝입니다. 코드도 안 바꾸고, 배포 구조도 안 바뀌요. 그래서 가장 먼저 꺼내는 카드입니다.

## 그렇다면 서버 성능을 계속 높이면 되지 않을까?

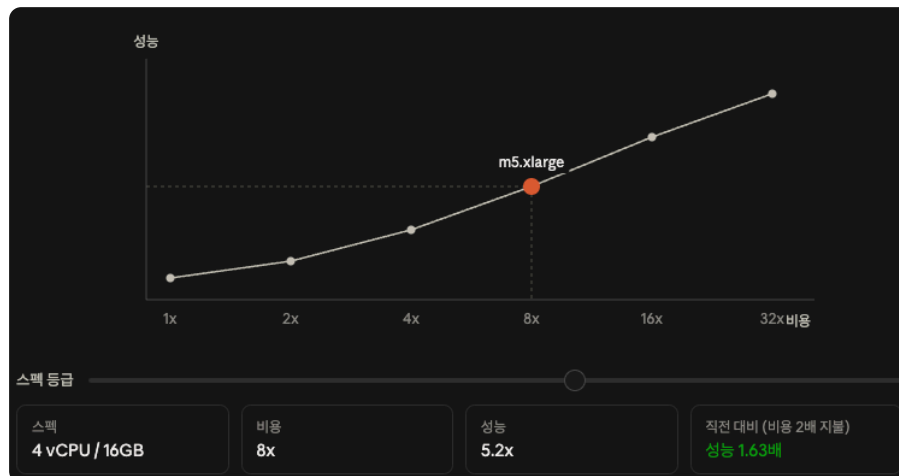
서버가 느려졌다고 해서 무조건 더 좋은 서버로 계속 바꾸면 해결될 것처럼 보일 수 있다.

실제로 **Scale Up** 은 가장 단순하고 빠른 확장 방법이다. 서버 대수는 그대로 두고, 기존 서버의 CPU, 메모리 같은 자원을 더 높은 사양으로 올리면 되기 때문이다.

하지만 여기에는 중요한 한계가 있다.

**서버 사양이 높아질수록, 비용은 계속 증가하지만 성능 향상은 점점 둔해지는 구간이 나타난다.**

즉, 처음에는 서버를 한 단계 올렸을 때 체감 성능이 꽤 좋아질 수 있지만, 어느 순간부터는 **더 많은 돈을 써도 기대만큼 성능이 오르지 않는다.**



예를 들어, 위 그래프처럼 **비용은 매번 2배씩 증가**하는데도 성능은 **1.8배 → 1.78배 → 1.63배 → 1.44배 → 1.27배** 처럼 점점 덜 늘어날 수 있다.

물론 이 수치는 **개념 이해를 위한 예시일 뿐**이며, 실제 성능 향상 폭은 서비스 구조와 워크로드에 따라 달라진다. 즉, 중요한 포인트는 특정 숫자가 아니라 **다음 사실**이다.

**Scale Up은 무한히 효율적이지 않다.**

어느 시점부터는 비용 대비 성능 효율, 즉 '가성비'가 급격히 떨어진다.

## Scale Up의 한계

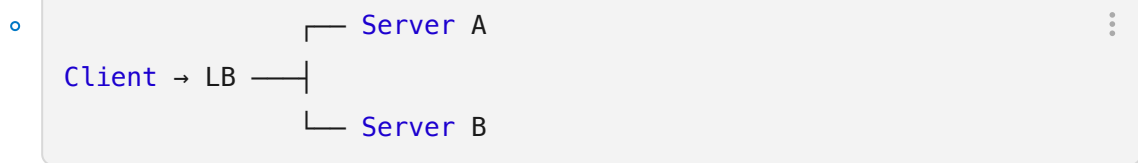
Scale Up은 구조를 크게 바꾸지 않고 빠르게 서버 성능을 높일 수 있다는 장점이 있지만, 명확한 한계도 존재한다.

1. 물리적 한계가 있다.

- 서버의 CPU나 Memory를 계속해서 높일 수 있는 것은 아니다.

2. 서버가 한 대 라면 **SPOF(Single Point of Failure)** 가 된다.

- 아무리 좋은 서버라도 1대가 죽으면 서비스 전체 중단
- 즉, Scale Up은 서버의 **성능 문제**는 해결할 수 있어도 서버 한 대에 의존한다는 **가용성 문제**까지 해결 해주지는 못한다.
- 그래서 실무에서는 트래픽이 많지 않은 서비스라고 해도 장애 대응을 위해 서버를 여러 대 구성하기도 한다.



이 구조에서는 Server A에 장애가 발생해도 Server B가 요청을 처리할 수 있다.

- 이처럼 일부 구성 요소에 장애가 발생하더라도 서비스를 계속 제공할 수 있도록 만드는 것을 **HA(High Availability, 고가용성)** 라고 한다.

**Scale Out**은 단순히 많은 트래픽을 처리하기 위한 방법만은 아니다.

서버를 여러 대 구성하면 **SPOF**를 줄이고, 장애 대응과 고가용성을 확보 할 수 있다는 장점이 있다.

### 3. APM (Application Performance Monitoring)

**APM**은 "서버가 느리다"를 발견하는 것에서 끝나는 게 아니라, "어떤 요청이, 어디에서, 왜 느려졌는지" 찾아가는 도구다.

#### 1. APM이 하는 일

APM은 "서버가 느리다"를 발견하는 도구가 아닙니다. 어떤 요청이, 어디에서, 왜 느려졌는지 찾아가는 도구입니다.

#### 2. 왜 Scale Up 다음에 배우는가?

Scale Up을 배웠다고 해서 이런 식으로 운영하면 안 된다.

서비스가 느림  
↓  
CPU가 높아 보임

↓

더 좋은 서버로 변경

먼저 해야 할 일은

서비스가 느림

↓

Monitoring / APM

↓

어디가 병목인가?

↓

원인에 맞는 해결책 선택

그리고 나서 아래와 같이 판단해야된다.

CPU 병목

→ Scale Up / Scale Out

DB Query 병목

→ Query Tuning / Index

Read 부하

→ Cache / Read Replica

외부 API 병목

→ Timeout / 비동기 처리 등

### 3. Tail Latency - 평균만 보지 않는다.

예를 들어 요청 10개가 이렇게 나왔다고 해봅시다.

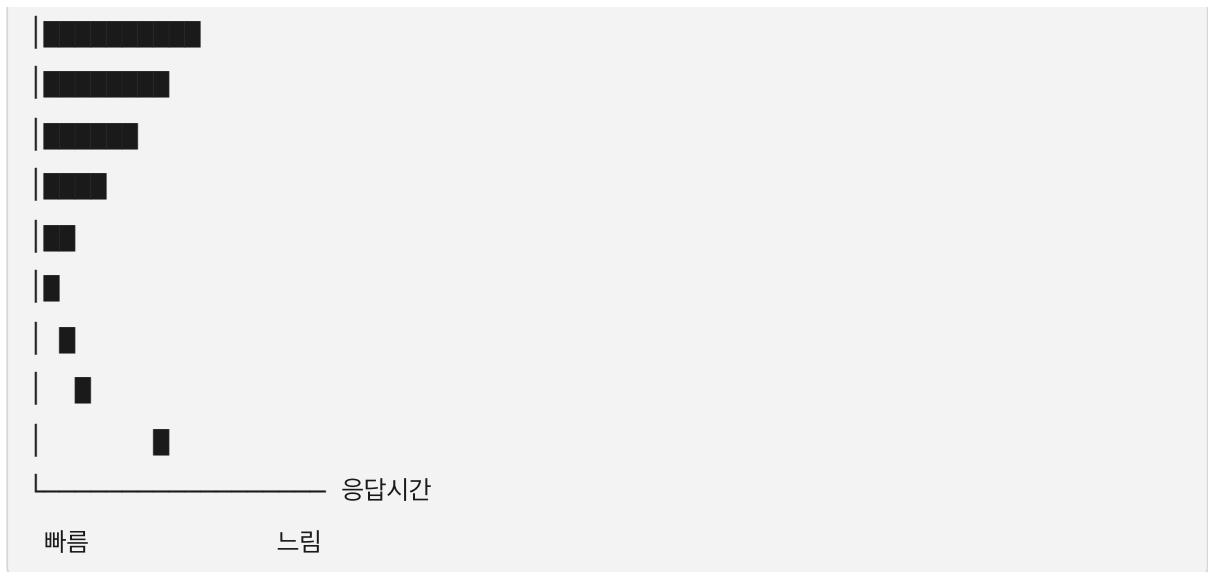
100, 110, 120, 100, 130, 110, 120, 100, 150, 5000 (ms)

대부분 100~150ms인데 하나가 5초가 나왔다.

웹 요청의 Latency는 항상 오른쪽으로 꼬리가 긴 분포가 되고, 이 꼬리 구간을 Tail Latency 라고 부릅니다.

요청 수

|



### Percentile 정의

| 지표  | 값 예시  | 의미                            |
|-----|-------|-------------------------------|
| p50 | 120ms | 약 50%의 요청이 120ms 이하. 중앙값과 유사  |
| p95 | 500ms | 약 95%가 500ms 이하. 나머지 5%는 더 느림 |
| p99 | 3.2s  | 약 99%가 3.2초 이하. 나머지 1%는 더 느림  |

### p99를 정확하게 읽는 법

- ✗ "100개 중 정확히 1개가 반드시 3.2초"
- ✓ "측정된 요청 중 약 99%가 3.2초 이하에 위치한다"

트래픽이 큰 서비스에서는 1%의 느린 요청도 수천~수만 건이 될 수 있기 때문에, 평균뿐 아니라 p95, p99 같은 Tail Latency를 함께 확인해야 한다.

### ★ 그렇다고 p99만 보는 것도 아니다

트래픽이 적은 서비스에서는 **p99가 몇 건의 요청에 크게 흔들립니다**. 하루 요청이 200건이면 p99는 상위 2건이 결정하니까요. 그래서 **Average**, **p50**, **p90**, **p95**, **p99**, **p99.9** 를 상황에 따라 함께 봅니다. 핵심은 평균을 버리는 게 아니라 평균만 보지 않는 것입니다.

## 4. 무엇부터 보는가 — Golden Signals와 RED

### Golden Signals (Google SRE)

시스템 전반을 볼 때 쓰는 네 가지입니다.

| 신호 | 질문 | 대표 지표 |
|----|----|-------|
|----|----|-------|

|                   |              |                                                          |
|-------------------|--------------|----------------------------------------------------------|
| <b>Latency</b>    | 얼마나 걸리는가     | Average, p95, p99                                        |
| <b>Traffic</b>    | 얼마나 들어오는가    | RPS, TPS                                                 |
| <b>Errors</b>     | 얼마나 실패하는가    | 5xx, Timeout, Exception                                  |
| <b>Saturation</b> | 한계에 얼마나 가까운가 | CPU 95%, Thread<br>200/200, Pool 30/30,<br>Queue 10,000건 |

## RED Method

웹 API에 특화된 축약판입니다. Spring 백엔드라면 이 세 개부터 봅니다.

Rate (요청량) → Errors (실패율) → Duration (응답시간)

두 가지 주의점

- **4xx를 5xx와 같이 묶지 마세요.** 401, 403, 404 는 서버 장애가 아닌 경우가 대부분입니다. 클라이언트가 잘못 요청한 거죠. 서버 장애를 볼 때는 **5xx Error Rate** 를 봅니다.
- **RPS와 TPS는 같은 말이 아닙니다.** RPS 는 초당 HTTP 요청 수이고, TPS 는 시스템이 정의한 트랜잭션의 초당 처리 수입입니다. 주문 1건이 API 호출 3번이면 RPS 3, TPS 1이 될 수 있어요. 문맥에 따라 다르니 확인하고 쓰세요.

## 4. Scale out

Scale Up이 서버 한 대의 성능을 높이는 방법이었다면, Scale Out은 서버의 개수를 늘려 처리 능력을 확장하는 방법이다.

초기 스타트업이 트래픽도 없는데 서버를 2대 두는 건 성능이 아니라 이것 때문입니다. **SP0F** 를 없애는 게 1차 목적이고, 처리량 증가는 따라오는 부수 효과에 가까워요.

면접에서 "왜 스케일아웃 했나요?"에 "트래픽이 늘어서요"만 답하면 절반입니다. "가용성 확보와 무중단 배포를 위해" 가 같이 나와야 합니다.

### LB가 하는 일 — 분산 말고 헬스체크

**Load Balancer** 를 "요청 나눠주는 애"로만 알면 절반만 아는 겁니다. 실무에서 더 중요한 건 **헬스체크**예요. LB는 주기적으로 각 서버에 **GET /health** 같은 요청을 보내고, **응답이 없는 서버는 라우팅 대상에서 자동으로 뺍니다.** 이게 있어야 "1대 죽어도 서비스가 산다"가 실제로 성립합니다. 죽은 서버로 계속 요청을 보내면 사용자 절반이 에러를 보게 되니까요.

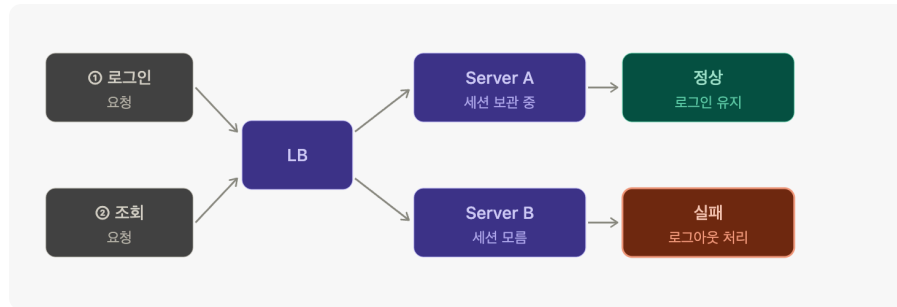
## Scale out의 중요한 전제 - Stateless

애플리케이션 서버를 Scale Out하려면 특히 중요한 조건이 있다.

어느 서버가 요청을 받더라도 동일하게 처리할 수 있어야 한다.

**Stateful** 하면 다른 서버에서 있는 내용이 다른 서버에서는 없을 수 있고, 다른 서버에서는 로그인 되어있는데 다른 서버에는 로그인이 세션이 없어서 로그인이 풀리는 상황을 경험하게 된다!

**Stateless** 는 서버가 특정 사용자의 상태를 자신의 Local Memory에 의존하지 않는 구조를 의미한다.



## DB 스케일아웃 (Master / Replica)

### 1. 문제 — DB는 앱 서버처럼 늘릴 수 없다

앱 서버는 아무것도 기억하지 않아서(stateless) 마음대로 늘렸습니다. DB는 반대입니다. **기억하는 것이 존재 이유**라서 그냥 복사해 늘리면 정합성이 깨집니다.

DB 1에 주문 저장 → DB 2에 조회 → 주문이 없다  
DB 1에서 잔액 차감 → DB 2에서도 차감 → 이중 차감

### 2. 해법 — 읽기와 쓰기를 다르게 취급한다

대부분의 서비스는 읽기가 압도적입니다. 손그림의 **7:3**, **8:2** 가 이 비율이에요.

| 서비스        | 읽기       | 쓰기        |
|------------|----------|-----------|
| 커머스 상품 페이지 | 수만 번 조회  | 그중 일부만 주문 |
| SNS 피드     | 수백 번 스크롤 | 글 작성 1번   |

그래서 **읽기만 여러 대로 나눠도 DB 부하의 70~80%가 해결됩니다.** 쓰기는 원본 한 곳으로 몰아 정합성을 지킵니다.

서버  
—— write (**INSERT** / **UPDATE** / **DELETE**) → Master (반드시 1대)  
—— read (**SELECT**) → Replica 1, Replica 2, ...

### 3. 용어

| 예전     | 현재 (권장)          | 역할         |
|--------|------------------|------------|
| Master | Primary / Writer | 쓰기 담당. 원본  |
| Slave  | Replica / Reader | 읽기 담당. 복제본 |

`master/slave` 는 업계에서 교체되는 중입니다. AWS 문서도 `primary / read replica` 표기. 면접에서는 둘 다 알아듣고, 말할 때는 `primary/replica` 를 쓰는 게 안전합니다.

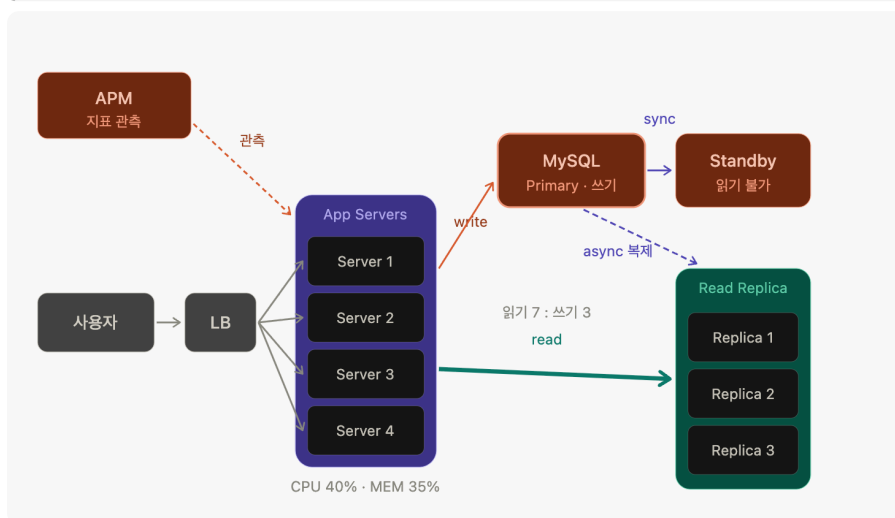
### 4. write 하나의 여정 — 이 챕터의 뼈대

| 단계 | 누가               | 무엇을                                   |
|----|------------------|---------------------------------------|
| ①  | 앱 서버             | Master에 write를 보낸다. Replica에는 보내지 않는다 |
| ②  | Master           | 커밋하고 즉시 응답한다. 여기서 서버의 일은 끝            |
| ③  | Master → Replica | 복제가 진행된다. DB끼리, 비동기로                  |

여기서 가장 중요한 것은 복제의 주체가 서버가 아니라 Master라는 점입니다. 서버는 ③이 언제 끝나는지 알지 못하고, 알 필요도 없습니다.

❌ "서버가 write할 때 Replica에 싱크를 맞춘다"

✅ "서버는 Master에만 write하고, Master가 Replica로 비동기 복제한다"



### 5. 복제의 정체 — 데이터 복사가 아니다

**복제** 라는 말이 오해를 만듭니다. 파일을 복사해 붙여넣는 게 아닙니다.

❌ 본사 장부를 사진 찍어 지사에 보낸다

✅ 본사가 "거래 내역서"를 보내고, 지사가 그걸 보고 자기 장부에 똑같이 적는다

그 거래 내역서가 **binlog(binary log)** 입니다. "회원 1001번 행의 이름을 김철수로 변경" 같은 **변경 이력**이 들어 있습니다.

받은 쪽이 그걸 **다시 실행**해야 하므로 당연히 시간이 걸립니다. 여기서 모든 게 따릅니다.

두 트랙이 따로 돈다

| 트랙      | 진행                                 |
|---------|------------------------------------|
| Master  | write 실행 → binlog 기록 → 커밋 → 응답. 끝  |
| Replica | binlog 수신 → relay log 저장 → 재생 → 반영 |

두 트랙이 서로를 기다리지 않는 것이 **비동기(async)**의 정확한 의미입니다.

## 6. 왜 기다리지 않는가 — 핵심 논리

동기로 만들면(Master가 Replica의 완료를 확인하고 커밋하면) 이렇게 됩니다.

| 상황              | 결과                  |
|-----------------|---------------------|
| Replica 1대가 느려짐 | 모든 write가 느려짐       |
| Replica 1대가 죽음  | <b>write가 멈춤</b>    |
| Replica를 추가     | <b>write가 더 느려짐</b> |

마지막 줄이 결정적입니다. 읽기를 확장하려고 Replica를 두는 건데, 늘릴수록 손해라면 목적 자체가 무너지니까.

**기다리지 않기 때문에 Replica를 10대, 20대로 늘릴 수 있습니다.** 복제 지연은 그 자유의 대가입니다.

백엔드에서 반복되는 패턴입니다. **확장성을 얻으려면 일관성을 조금 포기한다.** STEP 7의 캐시와 메시지 큐도 정확히 같은 거래입니다.

## 7. 복제 지연 (Replication Lag)

Master는 끝났는데 Replica는 아직 안 끝난 시간입니다.

| 시점      | 지연      |
|---------|---------|
| 평상시     | 5~50ms  |
| 트래픽 피크  | 1~5초    |
| 대량 배치 중 | 수십 초 이상 |

## 지연이 커지는 이유

Master는 여러 코어로 병렬 처리하지만, Replica는 binlog를 **순서대로 재생**합니다. Master가 초당 1,000건 처리하는데 Replica가 600건만 재생하면 매초 400건씩 밀리고, 밀린 게 누적됩니다.

대량 UPDATE를 돌리는 순간 서비스가 이상해지는 이유가 이것입니다. 그래서 배치는 새벽에, 나눠서 돌립니다.

## 8. 지연이 만드는 문제와 해결

① 글 작성 → Master 저장 → ② "완료!" → 사용자에게 응답  
↓ (③ 아직 진행 중)  
목록 조회 → Replica → 방금 쓴 글이 없다

서버는 "저장했다"고 믿는데 Replica는 아직 모르는 상태입니다. **개발 환경에서는 재현되지 않고 운영에서만 간헐적으로 터집니다.**

| 해결 방법                | 내용                            |
|----------------------|-------------------------------|
| Read-your-own-writes | 쓰기 직후 조회는 Master로 라우팅. 가장 일반적 |
| 낙관적 업데이트             | 화면에 먼저 반영. 프론트 경험 활용 가능       |

### Q. 복제 지연으로 어떤 문제가 생기나요?

쓰기 직후 Replica에서 조회하면 방금 쓴 데이터가 보이지 않을 수 있습니다. 저는 쓰기 직후 조회를 Master로 라우팅하는 read-your-own-writes 방식으로 처리하고, ReplicaLag 지표를 모니터링해 임계치를 넘으면 알림을 받도록 구성했습니다.

## 요약 — 다섯 줄

1. 앱 서버는 **Master에만 쓴다**. Replica에는 쓰지 않는다
2. Master는 커밋하고 **즉시 응답한다** (복제 완료를 기다리지 않음)
3. 복제는 **DB끼리, binlog를 전달해 비동기로** 진행된다
4. 그래서 **복제 지연**이 생기고, "방금 쓴 게 안 보임" 문제가 발생한다
5. 쓰기 직후 조회는 **Master에서 읽어서** 해결한다

## STEP 5 — Multi-Master를 실무에서 쓰지 않는 이유

### 1. 출발점 — 이 질문에서 시작한다

STEP 4-2의 결론은 이랬습니다.

읽기가 부족하다 → Replica 추가 → 해결 

쓰기가 부족하다 → Replica 추가 → 전혀 해결 안 됨 

그러면 자연스러운 아이디어가 떠오릅니다. "앱 서버를 여러 대 둔 것처럼, Master도 여러 대 두면 되지 않나?" 이론상 완벽해 보입니다. 쓰기를 A와 B로 나누면 쓰기 처리량이 2배가 되죠. 이것 **Multi-Master** 라고 합니다.

Master A ←양방향 복제→ Master B

▲ write                      ▲ write

그런데 실무에서는 거의 쓰지 않습니다.

## STEP 6 — DB 샤딩 (Sharding)

### 1. 출발점 — 여기까지 온 이유

지금까지의 결론을 정리하면 이렇습니다.

| 시도           | 결과                |
|--------------|-------------------|
| Read Replica | 읽기는 해결. 쓰기는 그대로   |
| Multi-Master | 쓰기 충돌로 실무에서 제외    |
| Master 스케일업  | 효과 있지만 물리적 상한 존재  |
| 캐시-메시지 큐     | 쓰기를 줄임. 근본 해결은 아님 |

전부 써봤는데도 부족하다면, 남은 방법은 하나입니다.

데이터 자체를 쪼개서 여러 DB에 나눠 담는다.

이게 **샤딩** 입니다. 쓰기를 진짜로 분산하는 유일한 방법이자, 복잡도가 가장 크게 뛰는 선택입니다.

### 2. 두 가지 분할 방향

먼저 용어를 구분해야 합니다. 데이터를 나누는 방향이 두 가지입니다.

#### 수직 분할 (Vertical Partitioning)

테이블·도메인 단위로 DB를 나눕니다.

통합 DB → 회원 DB | 주문 DB | 상품 DB

- 비교적 쉽고, 도메인 경계가 명확해짐

- STEP 8의 **MSA와 자연스럽게 이어짐**
- 한계: 주문 테이블 하나가 커지는 문제는 해결 못 함

수평 분할 (Horizontal Partitioning = 샤딩)

같은 테이블의 행을 여러 DB로 나눕니다.

주문 테이블 1억 건 → Shard 0 (3천만) | Shard 1 (3천만) | Shard 2 (4천만)

- 진짜 확장. 테이블 크기 문제를 해결
- 복잡도가 크게 상승
- 보통 "샤딩"이라고 하면 이쪽

실무에서는 수직 분할을 먼저 하고, 그래도 안 되면 수평 분할을 합니다.

### 3. 샤드 키 — 설계의 핵심

어떤 컬럼을 기준으로 나눌지 결정하는 것이 **샤드 키(Shard Key)** 입니다. **여기가 샤딩 설계의 90%입니다.**

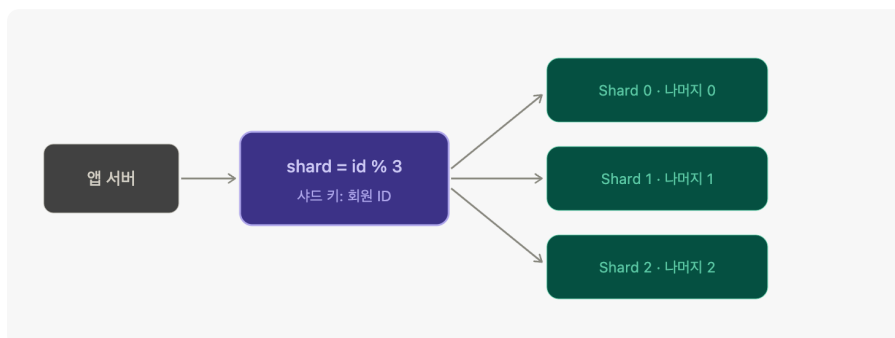
| 좋은 샤드 키의 조건  | 이유                        |
|--------------|---------------------------|
| 값이 골고루 분포    | 한 샤드에 몰리면(핫스팟) 샤딩한 의미가 없음 |
| 조회 조건에 자주 등장 | 없으면 모든 샤드를 다 뒤져야 함        |
| 변경되지 않음      | 바뀌면 데이터를 다른 샤드로 옮겨야 함     |

**user\_id**, **tenant\_id** 같은 것이 대표적입니다.

**잘못된 예:** 가입 날짜를 샤드 키로 쓰면, 최근 가입자가 몰린 샤드만 터집니다. 오래된 샤드는 놀고 있고요.

### 4. 손그림의 % 0 % 1 % 2 — 모듈로 해시 샤딩

네 번째 손그림이 이 방식입니다.



```

user_id = 1001 → 1001 % 3 = 2 → Shard 2
user_id = 1002 → 1002 % 3 = 0 → Shard 0
user_id = 1003 → 1003 % 3 = 1 → Shard 1

```

계산이 애플리케이션에서 일어난다는 점을 기억하세요. DB가 알아서 나눠주는 게 아니라, 서버가 "이 요청은 몇 번 샤드로 보낼지" 직접 판단합니다. 그래서 샤딩은 인프라 작업이 아니라 **애플리케이션 설계 작업**입니다.

장점은 명확합니다. 분포가 매우 균일하고, 계산이  $O(1)$ 로 빠릅니다.

그런데 치명적인 약점이 하나 있습니다.

## 5. 리샤딩 문제 — 샤딩의 최대 난제

샤드가 부족해서 3개 → 4개로 늘린다고 해봅시다. 그러면 % 3 이 % 4 로 바뀝니다. 직접 움직여보세요.



샤드 하나를 추가했을 뿐인데 **데이터의 75%가 다른 샤드로 이사해야 합니다.**

이게 무슨 의미인지 실제 규모로 환산해보면요.

데이터 3억 건, 샤드 3개 → 4개

이동 대상: 약 2억 2천만 건

이동하는 동안:

- 어느 샤드에서 읽어야 하는지 불확실 (구 위치? 신 위치?)
- 이동 중 들어오는 쓰기는 어디로?
- 며칠~몇 주가 걸리는 작업
- 실패하면 롤백도 지옥

그래서 실무에서 샤드 개수를 늘리는 건 **"프로젝트"**입니다. 스케일아웃처럼 버튼 하나로 되는 게 아니에요. 샤딩을 최후의 수단으로 두는 가장 큰 이유가 이것입니다.

## STEP 7-1 — Redis (캐시)

### 1. 왜 필요한가

STEP 1의 결론을 떠올려보면 이랬습니다. 처리량은 결국  $\text{커넥션 수} \div \text{쿼리 시간}$ .

그러니 가장 좋은 방법은 DB에 아예 안 가는 것입니다. 캐시가 하는 일이 정확히 이것입니다.

캐시 없음: 요청 100만 건 → DB 조회 100만 번

캐시 있음: 요청 100만 건 → DB 조회 1번, 나머지는 캐시

## 2. 왜 빠르냐

| 이유           | 설명                                 |
|--------------|------------------------------------|
| 메모리에 있다      | 디스크는 ms, 메모리는 마이크로초 단위. 수십~수백 배 차이 |
| Key-Value 조회 | 인덱스 탐색·조인·정렬이 없음. O(1)             |
| 단순한 실행 모델    | 단일 스레드 이벤트 루프라 락 경쟁이 없음            |

## 3. 쿠팡·배민 메인 페이지는 왜 빠르냐

배우신 그 질문의 답입니다. 메인 페이지 한 장은 사실 엄청나게 무거운 쿼리 묶음입니다.

추천 상품 + 인기 검색어 + 개인화 배너 + 카테고리별 베스트 + 실시간 랭킹 ...

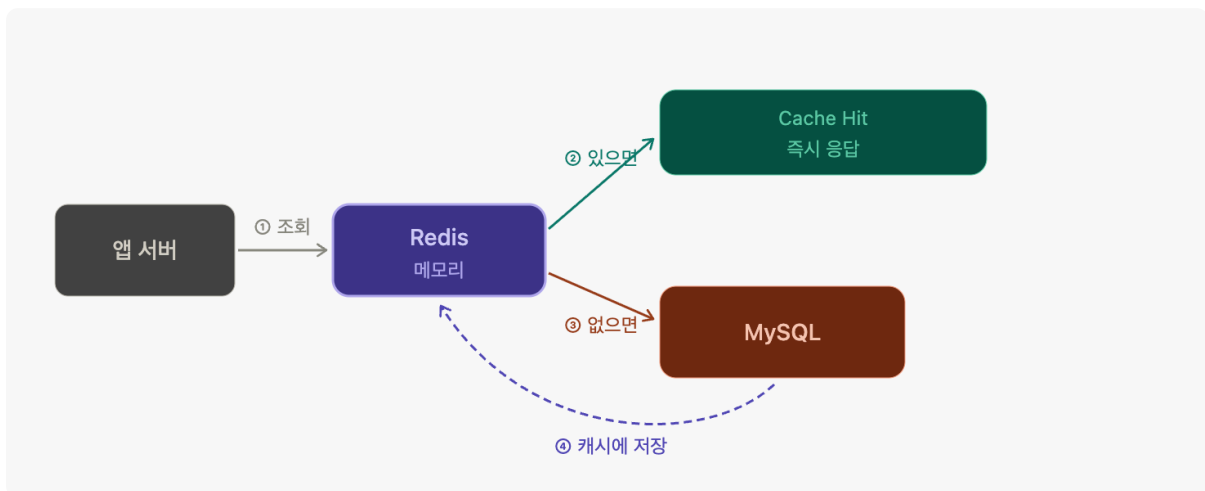
접속할 때마다 이걸 다 계산하면 절대 못 버팁니다. 그래서 실제로는 이렇게 합니다.

[배치] 새벽에 or N분마다 → 무거운 집계 쿼리 실행 → 결과를 Redis에 미리 넣어둠

[사용자 접속] → Redis에서 완성된 결과 1번 조회 → 응답 (DB 조회 0회)

핵심은 "빠르게 계산한다"가 아니라 "이미 계산해 둔 걸 꺼낸다" 입니다.

메인 페이지는 모든 사용자에게 거의 같은 화면이라 이 전략이 잘 통합니다. 반대로 "내 주문 내역"처럼 사용자마다 다른 데이터는 캐시 효율이 떨어집니다.



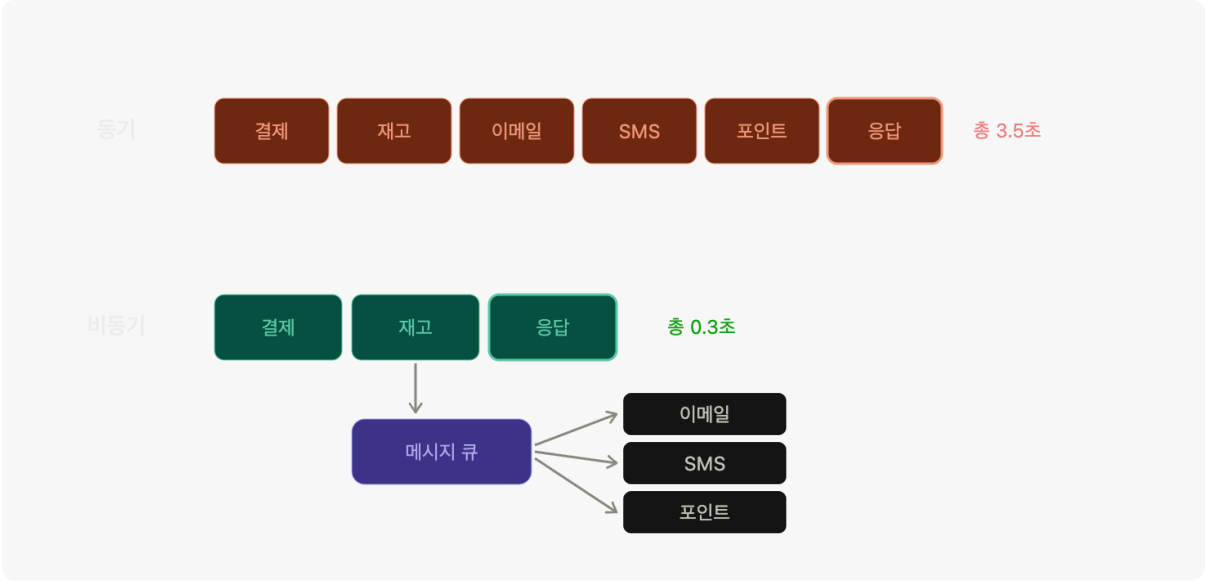
## STEP 7-2 — 비동기 메시지 큐

### 1. 문제 — 사용자가 왜 기다려야 하나

주문 하나를 처리할 때 서버가 하는 일을 나열해보면 이렇습니다.

결제 → 재고 차감 → 확인 이메일 → SMS 발송 → 포인트 적립

여기서 사용자에게 정말 필요한 건 앞의 두 개입니다. 이메일이 3초 후에 도착해도 아무도 불만이 없죠. 그런데 동기로 처리하면 사용자는 전부 끝날 때까지 기다립니다.



급하지 않은 작업을 응답 경로에서 빼내는 것이 메시지 큐의 본질입니다. 서버는 "이 일을 해달라"는 메시지만 큐에 넣고(발행), 바로 응답합니다. 뒤에서 별도 워커가 꺼내서(소비) 처리합니다.

대표 제품

| 제품            | 특징                             |
|---------------|--------------------------------|
| Kafka         | 대용량 스트리밍. 메시지를 보관해서 나중에 재처리 가능 |
| RabbitMQ      | 전통적 브로커. 라우팅 규칙이 유연            |
| AWS SQS       | 관리형. 운영 부담 최소                  |
| Redis Streams | 이미 Redis를 쓰고 요구가 단순할 때         |

Q. 메시지 큐를 왜 도입했나요?

주문 완료 후 이메일과 SMS 발송처럼 사용자가 기다릴 필요 없는 작업을 응답 경로에서 분리했습니다. 응답 시간이 줄었고, 외부 발송 서비스에 장애가 생겨도 주문 자체는 성공하도록 결합도를 낮출 수 있었습니다.

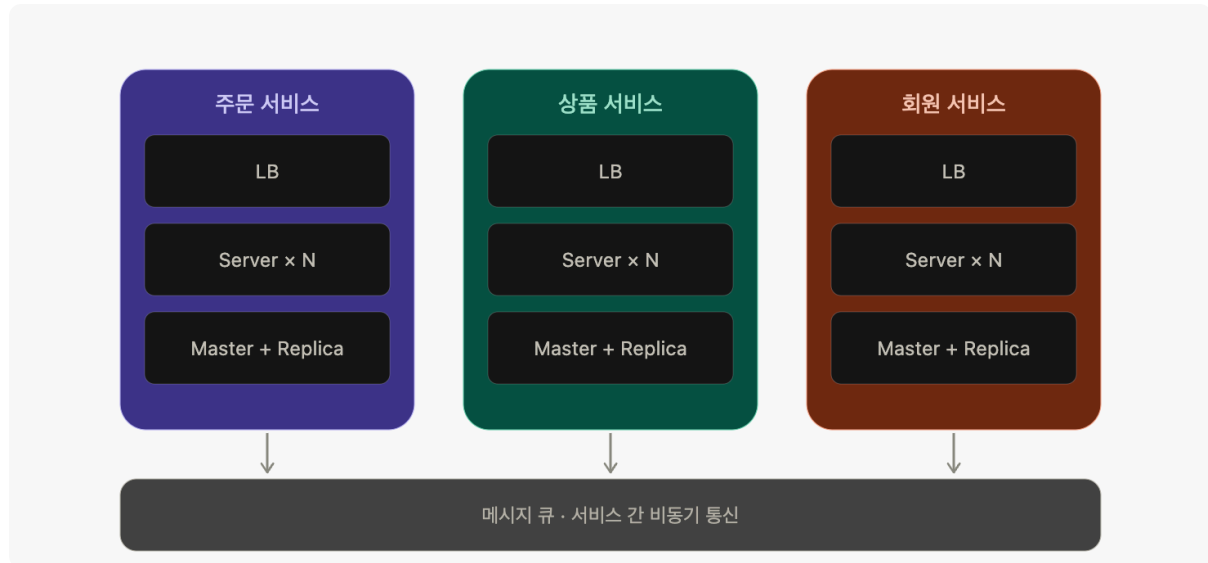
STEP 8 — 이 구조가 MSA의 한 단위

## 1.지금까지 만든 것

다섯 번째 손그림의 분홍색 큰 박스 전체가 하나의 세트입니다.

LB → 서버 N대 → Redis → Master/Replica → 메시지 큐

STEP 1부터 7까지 쌓아온 게 정확히 이 한 덩어리예요. **MSA(Microservice Architecture)** 는 이 세트를 도메인별로 여러 개 두는 구조입니다.



## 2. 핵심 원칙

| 원칙                          | 내용                                        |
|-----------------------------|-------------------------------------------|
| <b>Database per Service</b> | 서비스마다 DB를 따로 갖는다. 다른 서비스의 DB를 직접 조회하지 않는다 |
| 통신은 인터페이스로                  | API 호출(동기) 또는 메시지 큐(비동기)                  |
| 독립 배포·확장                    | 주문 서비스만 서버 10대로 늘릴 수 있다                   |
| 장애 격리                       | 상품 서비스가 죽어도 로그인은 된다                       |

첫 줄이 가장 중요하고, 가장 많이 어겨집니다. "급하니까 상품 DB 직접 조회하자" 를 한 번 허용하면 MSA가 아니게 됩니다.

**팀이 작으면 모놀리식이 정답입니다.**

MSA는 "조직이 커져서 팀별로 독립 배포가 필요할 때" 꺼내는 카드입니다. 기술적 이유보다 조직적 이유가 큼니다.

Scale out

7: (3)  
8: (2)

CPU 40%  
mem 35%

read

write

MySQL

k21

